

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Навчально-науковий інститут комп'ютерних наук та інформаційних технологій
Кафедра комп'ютерної інженерії та програмування

ЗВІТ
з лабораторної роботи № 1
з дисципліни «Програмування»

Виконала: студентка групи КН-9266
Компанієць Світлана Костянтинівна
Перевірів: доцент кафедри КІП
Лисиця Дмитро Олександрович

Харків - 2026

Тема: Вступ до програмування. Освоєння командної строки Linux

Мета роботи: налаштувати робоче середовище в ОС Linux, встановити необхідні пакети розробки, ознайомитися з основними командами терміналу, системою контролю версій Git, утилітою автоматизації збірки Make та виконати базові зміни у проєкті.

Завдання

Завдання 1. Підготовка системи для роботи з Linux (пропуск VirtualBox).

Підготувати комп'ютер до встановлення операційної системи Linux. Оскільки замість використання віртуальної машини VirtualBox було обрано рішення встановити Linux як другу операційну систему безпосередньо на ПК поруч із Windows 11 (режим Dual Boot), необхідно провести підготовчі дії в існуючій системі та звільнити місце на диску.

Завдання 2. Завантаження, встановлення та налаштування Debian-подібної ОС Linux.

Завантажити ISO-образ Debian-подібної операційної системи Linux (x64), встановити її на ПК поруч із Windows 11, налаштувати завантажувач для вибору операційної системи при включенні комп'ютера та оновити всі компоненти системи до актуальних версій через термінал.

Завдання 3. Встановлення базового набору пакетів розробки.

Інсталювати за допомогою терміналу та пакетного менеджера apt-get мінімально необхідні пакети програм для подальшої роботи: git, gcc, clang, clang-format, clang-tidy, tree, make, cppcheck. Після цього перевірити коректність їх встановлення та визначити отримані версії утиліт.

Завдання 4. Клонування репозиторію за допомогою Git.

За допомогою системи контролю версій Git та команди git clone виконати клонування навчального проєкту (репозиторію) з мережі Інтернет за посиланням, наданим викладачем (https://github.com/davydov-vyacheslav/sample_project.git).

Завдання 5. Ознайомитися з утилітою tree.

У терміналі за допомогою команди cd laboratory_works переходимо в робочу директорію проєкту та вводимо команду tree для відображення структури файлів і папок.

Завдання 6. Збірка, статичний аналіз та запуск проєкту за допомогою утиліти Make.

Перейти до директорії проєкту, де розташований файл Makefile, перевірити його наявність за допомогою команди ls, виконати автоматизовану збірку, очищення та

перевірку проєкту командою `make clean prep compile check`. Після цього за допомогою утиліти `tree` знайти згенеровані вихідні файли, перейти до каталогу `dist` та виконати отриманий бінарний файл.

Завдання 7. Виконати (обґрунтовано) будь-які зміни до коду, але при умові, що наступна компіляція проєкту буде виконана без помилок та зміни можна буде побачити у даних, що виводяться на екран проєктом.

Завдання 8. Додати до `Makefile` ціль `all`, яка послідовно виконує `clean`, `prep`, `compile` та `check`. Перевірити її роботу командою `make all`. Переконуватися, що файл `test.bin` не створюється, і перевірити роботу оновленої конфігурації.

Завдання 9. Визначити поточну версію утиліти `clang` та `make`.

Завдання 10. Дослідити роботу утиліти `man` та описати її призначення.

Завдання 11. За допомогою команди `git diff` показати виконані зміни у файлах.

Хід роботи

1. Для безпечного виконання робіт у Windows 11 було попередньо створено точку відновлення системи та вимкнено шифрування BitLocker, щоб уникнути можливих проблем із доступом до даних. Після цього за допомогою системного додатка «Керування дисками» (Disk Management) зменшено розмір основного тому Windows та виділено незайнятий простір обсягом близько 50 ГБ. У результаті систему Windows 11 було повністю підготовлено до встановлення другої ОС, а на накопичувачі отримано необхідний простір для розміщення Linux.

2. Було завантажено x64 ISO-образ Pop!_OS 24.04 LTS та утиліту balenaEtcher, за допомогою якої створено завантажувальну USB-флешку. У налаштуваннях BIOS/UEFI вимкнено параметр Secure Boot та виконано завантаження ПК з підготовленого носія. Під час встановлення Pop!_OS дисковий простір накопичувача обсягом 232.9 ГБ було розподілено у співвідношенні близько 78% до 22%: під Windows 11 залишено 181.3 ГБ, а під Pop!_OS виділено 50 ГБ (із розділенням на 39 ГБ під системний розділ `/`, 10 ГБ під `swap` та 1 ГБ під `/boot/efi`). Для налаштування меню Dual Boot у терміналі з правами `root` за допомогою команди `lsblk` було визначено EFI-розділ Windows (`/dev/sda1`), замонтовано його до тимчасово створеної точки `/mnt/windowsEFI` та скопійовано каталог Microsoft до

/boot/efi/EFI/. Далі через текстовий редактор nano відредаговано конфігураційні файли завантажувача в директорії /boot/efi/loader/, що дозволило сформувати меню вибору операційної системи при старті ПК. Наприкінці виконано оновлення пакетів системи командами `sudo apt-get update` та `sudo apt-get upgrade`.

У результаті на комп'ютер успішно встановлено Pop!_OS 24.04 LTS поруч із Windows 11, налаштовано завантажувач systemd-boot (рис.1) для зручного вибору ОС при включенні та оновлено всі системні компоненти до найновіших версій.



Рисунок 1 - Меню Dual-Boot з вибором між системами

3. У терміналі Pop!_OS за допомогою пакетного менеджера apt-get із правами суперкористувача (sudo) було виконано послідовне встановлення всіх необхідних пакетів за допомогою команд вида `sudo apt-get install -y <назва_пакета>`. Для перевірки успішності інсталяції та виведення інформації про версії програм у командному рядку було виконано команди з прапором `--version` для кожного інструмента. У результаті виконання команд було підтверджено успішне встановлення всього необхідного комплексу програмного забезпечення з такими версіями: Git версії 2.43.0, компілятор GCC версії 13.3.0, компілятор Clang версії 18.1.3, утиліта візуалізації каталогів Tree версії 2.1.1, утиліта автоматизації збірки GNU Make версії 4.3 та статичний аналізатор коду Cppcheck версії 2.13.0.

```
git version 2.43.0
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO warranty; not
even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Ubuntu clang version 18.1.3 (1ubuntu1)
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/bin
tree v2.1.1 © 1996 - 2023 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sesser,
Kyosuke Tokoro
GNU Make 4.3
Built for x86_64-pc-linux-gnu
```

Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Cpptest 2.13.0

4. У терміналі Pop!_OS було виконано команду `git clone https://github.com/davydov-vyacheslav/sample_project.git`. У процесі виконання утиліта Git підключилася до віддаленого сервера GitHub, завантажила всі файли проєкту, створивши в поточній директорії локальну копію репозиторію у папці `sample_project` (рис. 2):

```
svitlana@pop-os:~$ git clone https://github.com/davydov-vyacheslav/sample_project.git
Cloning into 'sample_project'...
remote: Enumerating objects: 214, done.
remote: Counting objects: 100% (64/64), done.
remote: Compressing objects: 100% (40/40), done.
remote: Total 214 (delta 32), reused 45 (delta 20), pack-reused 150 (from 1)
Receiving objects: 100% (214/214), 441.75 KiB | 3.45 MiB/s, done.
Resolving deltas: 100% (100/100), done.
svitlana@pop-os:~$
```

Рисунок 2 - Процес клонування віддаленого репозиторію

5. У терміналі Pop!_OS виконано перехід у папку проєкту за допомогою команди `cd sample_project`. Після цього було запущено утиліту `tree`, яка відобразила ієрархічне дерево всіх підкаталогів та файлів, що входять до складу клонованого репозиторію. У результаті було отримано наочне графічне представлення структури проєкту в терміналі (рис. 3).

```
svitlana@pop-os:~$ cd sample_project
svitlana@pop-os:~/sample_project$ tree
.
├── CMakeLists.txt
├── lab00
│   ├── CMakeLists.txt
│   ├── doc
│   │   ├── assets
│   │   │   └── animal-fields.png
│   ├── lab00.docx
│   ├── lab00.md
│   └── lab00.pdf
├── Doxyfile
├── Makefile
├── README.md
├── src
│   ├── lib.c
│   ├── lib.h
│   └── main.c
├── lab-cpp00
│   ├── CMakeLists.txt
│   ├── leaks_suppr.txt
│   ├── Makefile
│   ├── README.md
│   ├── src
│   │   ├── lib.cpp
│   │   ├── lib.h
│   │   └── main.cpp
│   └── test
│       └── test.cpp
├── README.md
├── sample_test_libcheck
│   └── test.c
└── 9 directories, 22 files
svitlana@pop-os:~/sample_project$
```

Рисунок 3 - Відображення структури каталогів за допомогою утиліти `tree`

6. У терміналі було виконано перехід до папки `~/sample_project/lab00`. За допомогою команди `ls` було перевірено та підтверджено наявність файла `Makefile` у каталозі. Далі з командою `make clean prep compile check` проведено збірку проєкту:

вилучено старі файли, створено папку dist та відкомпільовано код за допомогою clang. Під час перевірки утиліта clang-tidy показала декілька попереджень через відсутність деяких заголовкових файлів у коді (для функцій printf, random, srand), але бінарний файл main.bin усе одно успішно створився в папці dist (рис.4) .

```
svitlana@pop-os:~/sample_project/lab00$ tree
.
├── CMakeLists.txt
├── dist
│   └── main.bin
├── doc
│   ├── assets
│   │   └── animal-fields.png
│   ├── lab00.docx
│   ├── lab00.md
│   └── lab00.pdf
├── Doxyfile
├── Makefile
├── README.md
└── src
    ├── lib.c
    ├── lib.h
    └── main.c

5 directories, 12 files
svitlana@pop-os:~/sample_project/lab00$ cd dist
svitlana@pop-os:~/sample_project/lab00/dist$ ./main.bin
Інформація про тварину №01: Собака: зріст = 92 см, маса = 69 гр.
Інформація про тварину №02: Свиня: зріст = 23 см, маса = 67 гр.
Інформація про тварину №03: Кіт: зріст = 46 см, маса = 82 гр.
Інформація про тварину №04: Кіт: зріст = 72 см, маса = 96 гр.
Інформація про тварину №05: Корова: зріст = 85 см, маса = 88 гр.
Інформація про тварину №06: Корова: зріст = 89 см, маса = 37 гр.
Інформація про тварину №07: Свиня: зріст = 34 см, маса = 126 гр.
Інформація про тварину №08: Корова: зріст = 14 см, маса = 53 гр.
Інформація про тварину №09: Собака: зріст = 124 см, маса = 23 гр.
Інформація про тварину №10: Свиня: зріст = 62 см, маса = 62 гр.
svitlana@pop-os:~/sample_project/lab00/dist$
```

```
svitlana@pop-os:~/sample_project$ cd sample_project
svitlana@pop-os:~/sample_project$ cd ~/sample_project/lab00
svitlana@pop-os:~/sample_project/lab00$ make clean prep compile check
rm -rf dist
mkdir dist
clang -std=gnu11 -g -Wall -Wextra -Werror -Wformat-security -Wfloat-equal -Wshadow -Wconversion -Wlogical-not-parentheses -Wnull-dereference -Wno-unused-variable -Wno-implicit-int-float-conversion -Werror=vla -I./src src/lib.c src/main.c -o ./dist/main.bin
clang-format --verbose --dry-run --error src/*
Formatting [1/3] src/lib.c
Formatting [2/3] src/lib.h
Formatting [3/3] src/main.c
clang-tidy src/*.c -checks=-readability-uppercase-literal-suffix,-readability-magic-numbers,-clang-analyzer-deadcode.DeadStores,-clang-analyzer-security.insecureAPI.rand
Error while trying to load a compilation database:
Could not auto-detect compilation database for file "src/lib.c"
No compilation database found in /home/svitlana/sample_project/lab00/src or any parent directory
fixed-compilation-database: Error while opening fixed database: No such file or directory
json-compilation-database: Error while opening JSON database: No such file or directory
Running without flags.
[1/2] Processing file /home/svitlana/sample_project/lab00/src/lib.c.
2134 warnings generated.
[2/2] Processing file /home/svitlana/sample_project/lab00/src/main.c.
4267 warnings generated.
/home/svitlana/sample_project/lab00/src/lib.c:36:33: error: no header providing "random" is directly included [misc-include-cleaner,-warnings-as-errors]
11 |         entity->height = (unsigned int)random() % INT8_MAX;
    |                                     ^
/home/svitlana/sample_project/lab00/src/lib.c:36:44: error: no header providing "INT8_MAX" is directly included [misc-include-cleaner,-warnings-as-errors]
11 |         entity->height = (unsigned int)random() % INT8_MAX;
    |                                     ^
/home/svitlana/sample_project/lab00/src/lib.c:44:3: error: no header providing "printf" is directly included [misc-include-cleaner,-warnings-as-errors]
11 |         printf("Інформація про тварину №02u: ", i + 1);
```

Рисунок 4 - Результат виконання збірки проєкту та запуск програми ./main.bin

За допомогою команди tree перевірено наявності згенерованого файла dist/main.bin. Далі виконано перехід у папку dist та запуск програми командою ./main.bin, після чого в терміналі відобразилися дані про 10 тварин (їх назва, зріст та маса).

У результаті проєкт успішно зібрано та запущено.

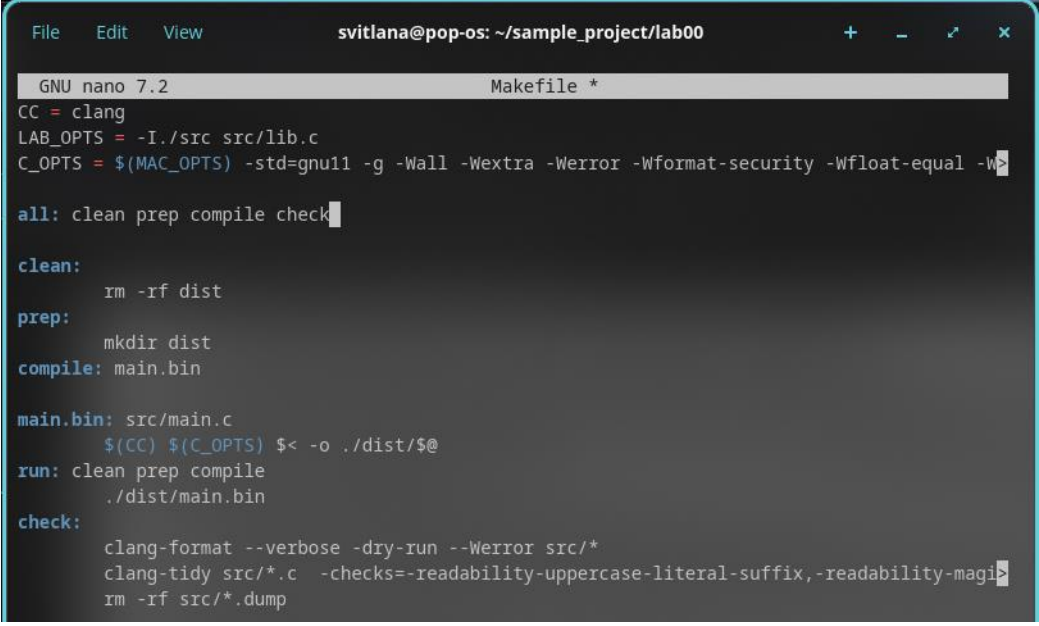
7. У текстовому редакторі nano відредагували файл src/lib.c. У коді замінено текстові назви тварин на кімнатні рослини (Фікус, Монстера, Заміокулькас, Сукулент), а вивід параметрів змінено на «висота» та «кількість листя». Після редагування виконано повторну збірку проєкту командою make clean prep compile. Компіляція пройшла успішно й без помилок. Під час запуску бінарного файла ./main.bin з папки dist у терміналі відобразилися оновлені дані про рослини.

У результаті зміни до коду внесено, проєкт успішно перезібрано, а оновлений вивід зафіксовано в консолі (рис.5).

```
svitlana@pop-os:~/sample_project/lab00$ nano src/lib.c
svitlana@pop-os:~/sample_project/lab00$ make clean prep compile
rm -rf dist
mkdir dist
clang -std=gnu11 -g -Wall -Wextra -Werror -Wformat-security -Wfloat-equal -Wshadow -Wconversion -Wlogical-not-parentheses -Wnull-de
reference -Wno-unused-variable -Wno-implicit-int-float-conversion -Werror=vla -I./src src/lib.c src/main.c -o ./dist/main.bin
svitlana@pop-os:~/sample_project/lab00$ cd dist
svitlana@pop-os:~/sample_project/lab00/dist$ ./main.bin
Інформація про рослину №01: Монстера: висота = 67 см, кількість листків = 58 шт.
Інформація про рослину №02: Заміокулькас: висота = 17 см, кількість листків = 17 шт.
Інформація про рослину №03: Кактус: висота = 69 см, кількість листків = 107 шт.
Інформація про рослину №04: Фікус: висота = 92 см, кількість листків = 56 шт.
Інформація про рослину №05: Кактус: висота = 20 см, кількість листків = 69 шт.
Інформація про рослину №06: Фікус: висота = 118 см, кількість листків = 17 шт.
Інформація про рослину №07: Заміокулькас: висота = 56 см, кількість листків = 30 шт.
Інформація про рослину №08: Монстера: висота = 122 см, кількість листків = 84 шт.
Інформація про рослину №09: Фікус: висота = 90 см, кількість листків = 7 шт.
Інформація про рослину №10: Монстера: висота = 108 см, кількість листків = 50 шт.
svitlana@pop-os:~/sample_project/lab00/dist$
```

Рисунок 5 - Заміна назв тварин на рослини

8. У редакторі nano додали до Makefile ціль `all: clean prep compile check`. У конфігурації вказано збірку `main.bin` замість `test.bin`. Усунули помилку форматування командою `clang-format -i src/lib.c`, після чого успішно виконали команду `make all`. Командою `ls dist` підтвердили створення файла `main.bin` та відсутність `test.bin`.



```
File Edit View svitlana@pop-os: ~/sample_project/lab00 + - ↵ ×
GNU nano 7.2 Makefile *
CC = clang
LAB_OPTS = -I./src src/lib.c
C_OPTS = $(MAC_OPTS) -std=gnu11 -g -Wall -Wextra -Werror -Wformat-security -Wfloat-equal -W
all: clean prep compile check
clean:
    rm -rf dist
prep:
    mkdir dist
compile: main.bin
main.bin: src/main.c
    $(CC) $(C_OPTS) $< -o ./dist/$@
run: clean prep compile
    ./dist/main.bin
check:
    clang-format --verbose -dry-run -Werror src/*
    clang-tidy src/*.c -checks=-readability-uppercase-literal-suffix,-readability-magi
    rm -rf src/*.dump
```

Рисунок 6 - Внесення змін до Makefile

9. У терміналі виконали команди `clang --version` та `make --version` та зафіксували виведені версії інструментів збірки та компілятора на рисунку 7.

```
svitlana@pop-os:~$ clang --version
Ubuntu clang version 18.1.3 (1ubuntu1)
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/bin
svitlana@pop-os:~$ make --version
GNU Make 4.3
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
svitlana@pop-os:~$
```

Рисунок 7 - перевірка версій утиліт clang та make у терміналі

10. Утиліта man (manual) призначена для перегляду довідки щодо команд, системних викликів і конфігураційних файлів у Linux. На практиці її використали для пошуку ключів і параметрів компілятора clang командою `man clang` (рис.8). За допомогою інтерактивного меню переглянули опис прапорців (наприклад, `-std=`, `-o`, `-I`), що дає змогу швидко знаходити потрібні ключі збірки без виходу в інтернет.

```
CLANG(1)                                Clang                                CLANG(1)

NAME
    clang - the Clang C, C++, and Objective-C compiler

SYNOPSIS
    clang [options] filename ...

DESCRIPTION
    clang is a C, C++, and Objective-C compiler which encompasses preprocessing, parsing, optimization, code generation, assembly, and linking. Depending on which high-level mode setting is passed, Clang will stop before doing a full link. While Clang is highly integrated, it is important to understand the stages of compilation, to understand how to invoke it. These stages are:

    Driver The clang executable is actually a small driver which controls the overall execution of other tools such as the compiler, assembler and linker. Typically you do not need to interact with the driver, but you transparently use it to run the other tools.

    Preprocessing
        This stage handles tokenization of the input source file, macro expansion, #include expansion and handling of other preprocessor directives. The output of this stage is typically called a ".i" (for C), ".ii" (for C++), ".mi" (for Objective-C), or ".mii" (for Objective-C++) file.
```

Рисунок 8 - Відкрита практичні довідка для компілятора clang за допомогою утиліти man

11. У директорії `~/sample_project/lab00` виконали команду `git diff`. Утиліта відобразила зміни в конфігураційному файлі Makefile (додавання цілі `all: clean prep compile check`) та відредаговані рядки у `src/lib.c` (рис. 9).

```
svitlana@pop-os:~/sample_project/lab00$ git diff
diff --git a/lab00/Makefile b/lab00/Makefile
index aedfa28..4239010 100644
--- a/lab00/Makefile
+++ b/lab00/Makefile
@@ -2,6 +2,8 @@ CC = clang
LAB_OPTS = -I./src src/lib.c
C_OPTS = $(MAC_OPTS) -std=gnu11 -g -Wall -Wextra -Werror -Wformat-security -Wfloat-equal -Wshadow -Wconversion -Wlogical-not-parentheses -Wnull-dereference -Wno-unused-variable -Wno-implicit-int-float-conversion -Werror=vla $(LAB_OPTS)

+all: clean prep compile check
+
+clean:
+    rm -rf dist
+
+prep:
diff --git a/lab00/src/lib.c b/lab00/src/lib.c
index f7f86e0..37fed54 100644
--- a/lab00/src/lib.c
+++ b/lab00/src/lib.c
@@ -2,8 +2,7 @@
 * @file lib.c
 * @brief Файл з реалізацією функцій оперування тваринами
 *
- * @author Davydov V.
- * @date 14-apr-2020
+ * @author Davydov V. * @date 14-apr-2020
 * @version 1.3
 */

@@ -14,16 +13,16 @@ const char *get_animal_type_name(enum animal_type type)
    const char *result;
    switch (type) {
        case CAT:
            ...skipping...
diff --git a/lab00/Makefile b/lab00/Makefile
index aedfa28..4239010 100644
--- a/lab00/Makefile
+++ b/lab00/Makefile
@@ -2,6 +2,8 @@ CC = clang
```

Рисунок 9 - Відображення внесених змін у Makefile

Контрольні питання

1. Що таке Операційна Система?

Які операційні системи ви знаєте? Операційна система (ОС) — це базовий комплекс програмного забезпечення, який керує апаратними ресурсами комп'ютера (процесором, оперативною пам'яттю, накопичувачами, пристроями вводу-виводу) та надає середовище для виконання користувацьких програм. Прикладами ОС є Pop!_OS, Ubuntu, Debian, Arch Linux, macOS, Windows 11, Android та iOS.

2. Які засоби віртуалізації ОС існують?

Існують засоби апаратної віртуалізації (гіпервізори), такі як VirtualBox, VMware Workstation / ESXi, KVM, Hyper-V і QEMU, а також засоби контейнеризації (віртуалізації на рівні ОС), серед яких Docker, Podman та LXC/LXD.

3. Що таке менеджер пакетів?

Які менеджери пакетів існують? Менеджер пакетів — це утиліта для автоматизації встановлення, оновлення, налаштування та видалення програмного забезпечення в системі, а також для автоматичного вирішення залежностей між ними. Серед відомих менеджерів пакетів можна виділити apt (Debian/Ubuntu/Pop!_OS), pacman (Arch Linux), dnf / yum (Fedora/RHEL), Flatpak, Snap, Homebrew (macOS/Linux), npm (Node.js) та pip (Python).

4. Що таке система контролю версій?

Які існують системи контролю версій? Система контролю версій (VCS) — це програмне забезпечення для фіксації, відстеження та управління змінами у вихідному коді або файлах проєкту з можливістю повернення до попередніх станів і паралельної роботи кількох розробників. До розподілених VCS належать Git та Mercurial, а до централізованих — Subversion (SVN) і Perforce.

5. Що таке Makefile? Які його ключові обов'язки? Як його використовувати?

Makefile — це текстовий файл із правилами та інструкціями для утиліти make, який описує порядок збірки проєкту з вихідних файлів. Його ключовими обов'язками є автоматизація компіляції та компонування, відстеження залежностей (перекомпіляція лише змінених файлів) та виконання рутинних завдань, як-от очищення директорій чи статичний аналіз. Його використовують шляхом запуску в терміналі команди make або make <ціль> (наприклад, make all, make clean).

6. Що робить команда tree?

Команда tree виводить у термінал графічне (деревоподібне) представлення структури каталогу, відображаючи всі вкладені папки та файли із дотриманням їхньої ієрархії.

7. Як дізнатися перелік файлів та даних, що були змінені за допомогою системи контролю версій git?

Для перегляду короткого списку модифікованих, доданих або видалених файлів використовують команду git status. Для перегляду конкретних текстових змін у файлах використовують команду git diff. Для аналізу історії збережених фіксацій (комітів) із деталізацією змін або списком зачеплених файлів застосовують команди git log -p або git log --stat.

Висновок. Під час лабораторної роботи підготовлено робоче середовище в Pop!_OS, встановлено інструменти розробки (Git, Clang, Make) та клоновано навчальний проєкт. Опановано збірку коду за допомогою утиліти make, внесено зміни до вихідного коду src/lib.c, налаштовано ціль all у Makefile та перевірено модифікації через git diff і довідку man. Завдання виконано повністю, програма компілюється та працює без помилок.